

PIAR: Path-Improved Adaptive Routing for Dragonfly Networks

Zhenghao Wang^{†§}, Qiang Wang^{†§}, Mingche Lai^{†*}, Jiaqing Xu[†], Jinbo Xu[†], Min Xie[†], Guo Chen[‡]

[†]College of Computer Science and Technology, National University of Defense Technology, Changsha, China

[‡]College of Computer Science and Electronic Engineering, Hunan University, Changsha, China

Email: [†]{zh_wang, qiangwang, mingchelai, xujiaqing, xujinbo, xiemin}@nudt.edu.cn, [‡]guochen@hnu.edu.cn

Abstract—For the next-generation exascale supercomputing communication systems, Dragonfly topology offers strong scalability, low latency, and cost efficiency. Dragonfly networks have already been implemented in current supercomputers and will continue to expand in future systems. Adaptive routing in Dragonfly topologies is critical for network performance. The traditional UGAL routing algorithm, which uses the valiant mechanism to select non-minimal paths, does not adequately consider the impact of high hops in non-minimal paths, often unnecessarily increasing the average path length, thereby increasing network latency and load. Furthermore, UGAL inaccurately estimates the congestion of the entire routing path based on local information, leading to suboptimal routing decisions that limit the algorithm's performance. In this paper, we propose PIAR, a novel path-improved adaptive routing algorithm. PIAR dynamically selects paths based on the status of local and global channels, prioritizing non-minimal paths with fewer hops to reduce network latency and load, thereby improving network performance. Additionally, we present the microarchitecture of the routing computation unit. Our evaluation results demonstrate that, compared with advanced algorithms such as PAR_{PH}, TPR, and UGAL_{LE}, PIAR achieves an average throughput improvement of 19.2% and reduces latency by up to 13.4% under the single synthetic traffic. Under mixed traffic, PIAR achieves an average throughput improvement of 23.6% and reduces the latency by up to 33.8%. For application workloads, PIAR achieves an average reduction of 24.0% in packet latency.

Index Terms—High-performance interconnection network; dragonfly topology; adaptive routing

I. INTRODUCTION

High-performance interconnection networks are critical infrastructure for high-performance computing (HPC) systems and intelligent computing centers. In recent years, the rapid advancement of supercomputers and the continuous growth of system scale [1] have posed unprecedented challenges concerning network cost, power consumption, and stability. Although the Fat Tree [2] topology offers favorable traffic balancing characteristics and high bisection bandwidth, its hierarchical scaling method results in significant scalability bottlenecks and cost challenges [3]. For example, the node scale of the Tianhe [4] and Sunway [5] supercomputers is limited to tens of thousands. In response to these challenges, new topologies based on high-radix routers [6] have emerged in recent years, such as Flattened Butterfly [7], HyperX [8], Dragonfly [9]–[11], Slimfly [12], Megafly [13], Galaxyfly [14],

and PolarFly [15]. Among these, the Dragonfly topology has gained significant traction due to its exceptional scalability and cost-efficiency, becoming the topology of choice for some of the world's leading supercomputing systems [16], such as Frontier and Aurora.

The Dragonfly topology provides a low-diameter network with high path diversity. However, achieving high-performance routing in a Dragonfly network is a complex and challenging task. Minimal routing (MIN) works well for uniform traffic but proves inadequate when dealing with complex traffic patterns, such as adversarial traffic patterns. If only static minimal routing is employed, the limited path choices cannot adapt to these complex traffic patterns, leading to congestion and a deterioration in network performance. To address this issue, routing schemes often incorporate Valiant Load Balancing (VLB) [17] routing, which introduces a significant number of non-minimal paths to leverage path diversity and alleviate network congestion. Universal Globally Adaptive Load-balanced routing (UGAL) [18], a classical load balancing routing algorithm suitable for arbitrary symmetric topologies, adaptively selects either MIN or VLB paths to relieve congestion. However, in practical implementations, UGAL typically relies on local information, such as local queue occupancy or credit counts, to approximate the entire routing path latency. This reliance on limited information often results in suboptimal routing decisions. Additionally, when selecting VLB paths, UGAL randomly chooses intermediate routers or groups. While this helps with load balancing, it can lead to an increase in the average path length, thereby causing higher network latency and load. This excessive consumption of resources negatively impacts the network's throughput performance.

To overcome these limitations, this paper proposes **Path-Improved Adaptive Routing (PIAR)** algorithm. PIAR dynamically selects paths based on the status of local channels and global channels, prioritizing non-minimal paths with fewer hops to reduce network latency and load. PIAR not only effectively mitigates network congestion but also significantly reduces both path length and resource consumption through more refined path selection. Specifically, the main contributions of this paper are the following:

- We identify the limitations of UGAL, particularly its insufficient consideration of the impact of high hop counts on non-minimal paths, where the increase in average path

[§]Both authors contributed equally to this work. ^{*}Mingche Lai is the corresponding author.

length leads to higher network latency and load.

- We propose Path-Improved Adaptive Routing (PIAR) algorithm. PIAR dynamically selects shorter and more efficient non-minimal paths based on local and global link congestion information. Additionally, we design an efficient and flexible microarchitecture of the routing computation unit.
- We evaluate the proposed PIAR algorithm and compare it with PAR_{PH}, TPR, and UGAL_LE algorithms. Under the single synthetic traffic, PIAR achieves an average throughput improvement of 19.2%, and reduces the latency by up to 13.4%. Under mixed traffic, throughput increases by an average of 23.6% and reduces the latency by up to 33.8%. Under application workloads, PIAR achieves an average latency reduction of 24.0%.

II. BACKGROUND

A. Dragonfly Topology

The Dragonfly topology is a widely adopted interconnection network topology in high-performance computing systems, offering advantages such as strong scalability, low diameter, and cost-effectiveness.

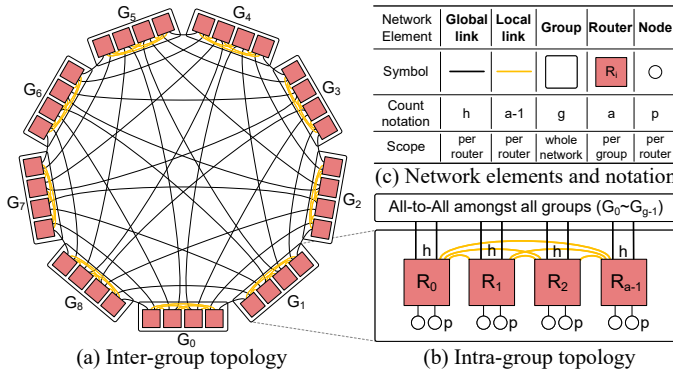


Figure 1: Hierarchical structure of the Dragonfly topology

As shown in Figure 1, the Dragonfly topology has a two-layer structure. Multiple routers/switches are interconnected via local links to form groups (which can be viewed as virtual routers). These groups are connected to each other through global links to form the entire network [9]. Specifically, the Dragonfly topology is defined by four key parameters: p , a , h , and g . Here, p represents the number of nodes connected to each router, and a indicates the number of routers within each group. For example, when a 1D flattened butterfly topology is used within a group, each router has $a - 1$ ports for intra-group interconnection. h represents the number of global links each router uses for inter-group connections. Therefore, the total number of ports (i.e., radix) for each router is $k = p + h + (a - 1)$. The parameter g represents the total number of groups. In the largest-size Dragonfly topology, each pair of groups is connected by a single global link, and the network scale satisfies $g = ah + 1$, enabling the system to scale to $(ah + 1)ap$ nodes.

Previous studies have proposed various global link arrangement schemes [19], including relative, absolute, and circulant-based arrangements. In addition, per-router arrangements [20] have also been considered. In this paper, we use $dfl(p, a, h, g)$ to represent the Dragonfly topology, where these parameters define the overall structure of the network. For instance, Figure 1 shows an example of the $dfl(2, 4, 2, 9)$ topology. Since each packet along its MIN path typically traverses two local channels and one global channel, the relationship $a = 2p = 2h$ [9] is typically satisfied in a load-balanced Dragonfly network. In a typical Dragonfly topology, groups are generally fully connected to ensure overall system connectivity, while there are various methods for intra-group connections, such as the 2D all-to-all mesh in the Cray Cascade architecture [11] and the tree structure in Dragonfly+ [10]. This study focuses on the Dragonfly topology with full intra-group connectivity.

B. Typical Routing Paths in Dragonfly

The typical routing paths in Dragonfly are shown in Figure 2. For intra-group traffic, the MIN path is denoted as **L**. The VLB path can be represented as **L-L**, where an intermediate R_i that is neither the source nor the destination is randomly selected by VLB routing. For inter-group traffic, the MIN path directly reaches the destination group G_d from the source group G_s , and is generally represented as **LGL** (two local channels L and one global channel G). While the MIN routing scheme is sufficient for uniform random traffic, it suffers under adversarial traffic patterns, where all traffic from a group is forwarded through a single global channel, resulting in a throughput bottleneck of $1/ap$. To handle this, VLB routing randomly selects an intermediate router R_i or an intermediate group G_i , neither of which belongs to the source or destination group, forwarding the packet first to R_i or G_i and then to the destination router. The VLB path is generally represented as **LGL-LGL**. While VLB paths increase path diversity under extreme traffic conditions, they also significantly increase overall latency.

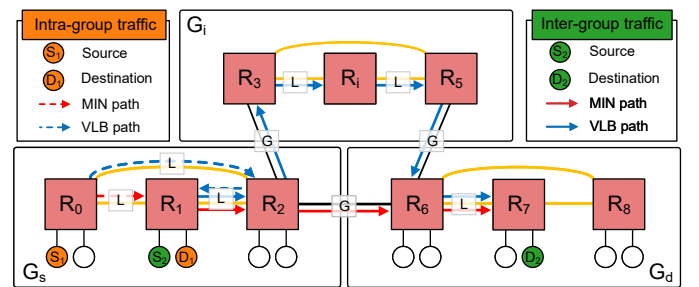


Figure 2: MIN and VLB paths in Dragonfly

To reduce the length of VLB paths, the following strategies can be employed: 1) select the intermediate group adjacent to the source router, and the path is represented as **GL-LGL**; 2) directly forwarding through the exit router within the intermediate group, represented as **LG-L-GL**. However, both strategies have limitations: the **GL-LGL** strategy is not suit-

able for adversarial-consecutive traffic (ADV_c) [21], where traffic is concentrated in a single router in the source group, making it difficult to detect congestion effectively; the **LG-L-GL** strategy is unsuitable for ADV+ h [22] with a group offset distance of h , where the intermediate group transmits packets through a single intra-group link, limiting throughput to $1/h$. Fixed path strategies fall short under dynamic traffic. An ideal adaptive routing algorithm should proactively mitigate congestion and reduce latency by selecting shorter paths.

C. Universal Globally Adaptive Load-balanced Routing

In the Dragonfly topology, UGAL [9] adaptively chooses between minimal (MIN) paths and non-minimal (VLB) paths, where the VLB path is constructed by randomly selecting an intermediate group or router, dividing the route into two segments of MIN paths. UGAL selects the routing path based on local queue occupancy or credit count; however, its performance is limited due to reliance solely on local information. Specifically, UGAL chooses the MIN path when the following condition is satisfied:

$$Q_m \times H_m \leq Q_{nm} \times H_{nm} + T \quad (1)$$

where Q_m and Q_{nm} denote the queue lengths or credit counts for MIN and VLB paths, respectively; H_m and H_{nm} represent the hop counts for the corresponding paths; and the threshold T biases UGAL toward either the MIN or VLB path.

III. RELATED WORK

A variety of routing algorithms and techniques have been proposed to enhance the network performance of Dragonfly topology. When Kim et al. first introduced the Dragonfly network [9], they introduced two versions of the UGAL algorithm: UGAL_L, which uses the local queue information from the current router, and UGAL_G, which uses the queue information of all global channels in G_s . Piggyback [23] employs Explicit Congestion Notification (ECN) to exchange global link information within a group, enabling precise path optimization decisions. Progressive Adaptive Routing (PAR) [23] algorithm can re-evaluates the UGAL decisions for packets within the source group, significantly improving performance compared to UGAL. Won et al. [24] further propose a method based on history window to address “phantom congestion” caused by PAR under high-latency link conditions. OFAR [22] can dynamically switch the packet routing from minimal paths to non-minimal paths in both intra-group and inter-group communications. OFAR-CM [25] further studies congestion management mechanisms. RLM/OLM [26] proposed two deadlock-free adaptive routing mechanisms, supporting non-minimal routing decisions at the local and global levels, and allowing each hop to dynamically update routing decisions. Contention-based nonminimal adaptive routing [27] utilizes counters to record port contention for routing decisions. TPR [28] infers intra-group and inter-group communication patterns by setting traffic counters, aiding more accurate routing decisions. T-UGAL [29] optimizes average

hop counts for VLB paths, thereby improving network performance. ACOR [30] distinguishes different VLB paths with different hops and prioritizes paths with fewer hops, but is limited to source routing. LIA [31] safely reduces the number of hops of non-minimum paths based on counters and adjusts routing decisions based on the obtained network information, but is also limited to source routing. DGB [32] proposes a gradient-based bias routing algorithm that dynamically sets the bias value T according to network congestion conditions. Q-adaptive [33] introduces a multi-agent reinforcement learning method to assist routing decisions. UGAL_ML [34] is a machine learning-based routing scheme that makes routing decisions similar to UGAL_G by utilizing local router information. UGAL_LE [35] achieves better performance by improving the accuracy of the latency estimation.

IV. PATH-IMPROVED ADAPTIVE ROUTING

This section first discusses the problems of UGAL and gives our motivation. Then we propose PIAR, which uses local and global channel congestion information to determine whether a path is available. PIAR distinguishes the length of VLB paths and prioritizes paths with fewer hops, optimizing routing decisions to alleviate network congestion. Finally, we introduce the microarchitecture of the routing computation unit.

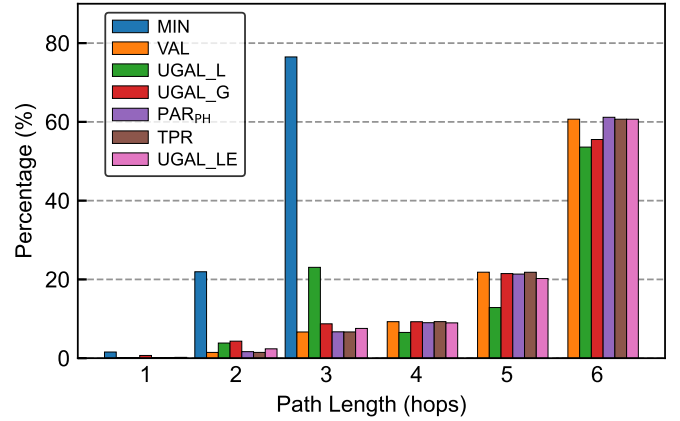


Figure 3: Path length distribution under the ADV traffic

A. Motivation

Routing algorithms play a key role in optimizing network performance, typically reflected in two aspects [36]: 1) achieving load balancing through path diversity, which reduces network congestion and improves channel utilization; 2) reducing the average routing hop count, which lowers network resource consumption. An ideal routing algorithm should balance the traffic distribution across network links, especially when dealing with non-uniform traffic patterns. Furthermore, path length locality is also one of the core objectives of routing algorithms. A routing algorithm should prefer shorter paths to improve network transmission efficiency.

Taking the UGAL routing algorithm as an example, UGAL adaptively selects MIN and VLB paths. In the case of low

load or balanced traffic, UGAL will select the MIN path; when the traffic is unbalanced, UGAL uses a VLB path with a higher number of hops to achieve load balancing. Nevertheless, UGAL does not distinguish between VLB paths when selecting them; instead, it randomly selects intermediate groups or routers to decide the candidate VLB paths. As a result, UGAL often uses 6-hop VLB paths, without fully utilizing shorter 3-hop, 4-hop, or 5-hop VLB paths. As shown in Figure 3, under the ADV traffic pattern, the results show the packet path distribution for different routing algorithms when the network approaches saturation, with a network scale of $dflly(4, 8, 4, 33)$. Under MIN routing, packets mainly select 1-hop to 3-hop MIN paths, with **LGL** paths accounting for 76.5%. In VAL routing, all paths are VLB paths, with typical **LGL-LGL** paths accounting for 60.7%. In UGAL_L and UGAL_G routing, **LGL-LGL** paths account for 53.6% and 55.5%, respectively, with average hop counts of 4.89 and 5.13. In contrast, in PAR_{PH}, TPR, and UGAL_LE, **LGL-LGL** paths account for more than 60%. Assuming that 50% of the 6-hop paths in UGAL_L are changed to 4-hop paths, the average packet hop count would decrease to 4.34, a reduction of 11.2%. This indicates an approximately 11% reduction in packet delay under low network load and an 11% decrease in network load near saturation.

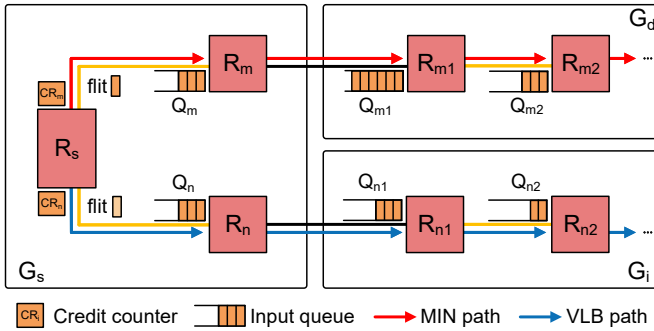


Figure 4: Inaccuracy of estimating global link congestion

In addition, UGAL estimates the channel load of global links based on local link queue lengths. This approximation method performs well under strong backpressure but may lead to higher latency in environments with soft backpressure. Figure 4 demonstrates the limitations of approximating global link queue loads Q_{m1} and Q_{n1} using local link queues Q_m and Q_n . In this scenario, even though local link queues Q_m and Q_n have not yet congested, the corresponding global links may already be congested. Only after a lot of time is spent filling up Q_{m1} , Q_m can reflect the congestion of Q_{m1} , so these packets will accumulate higher latency during this period. The inaccuracy in estimating global link load from local link queue length causes UGAL to make suboptimal routing decisions.

B. PIAR

The key insight behind PIAR is to prioritize shorter paths under non-congested conditions in order to optimize packet latency and improve network throughput performance.

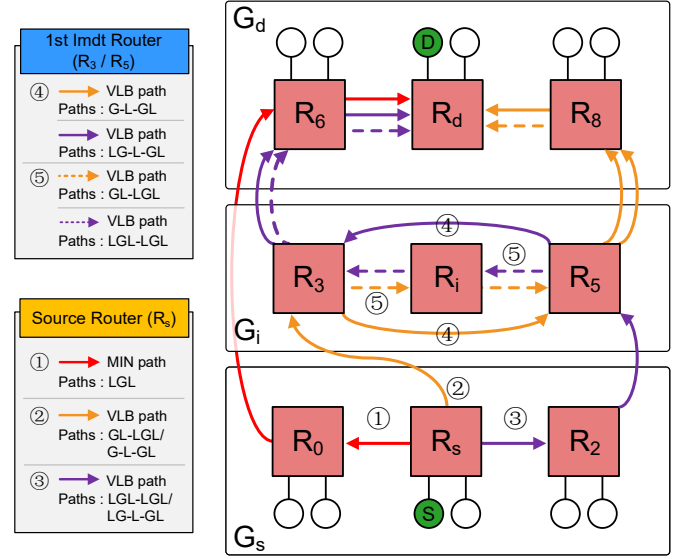


Figure 5: Classification of inter-group routing paths in Dragonfly

1) *Path Classification:* In the Dragonfly topology, we denote a node $C_i = \langle R_i, G_i \rangle$ as being attached to router R_i within group G_i . Given a source node $C_s = \langle R_s, G_s \rangle$ and a destination node $C_d = \langle R_d, G_d \rangle$, multiple routing paths exist between them, each corresponding to a different number of hops, as shown in Figure 5. For the inter-group traffic, in the source group, the paths can be categorized into three scenarios. In scenario ①, the MIN path from router R_s reaches R_0 and directly connects to the destination group G_d via the global link, then proceeds to R_d , with the fewest hops. Usually the MIN path is **LGL**, but there are also **G/LG/GL** paths. In scenario ②, the VLB path goes from R_s 's global port to the intermediate group G_i , then reaches G_d . This path is typically **GL-LGL/G-L-GL**, skipping one local hop in the source group. In scenario ③, the VLB path goes through router R_2 in the source group to reach G_i , and then to G_d , with the path typically being **LG-L-GL/LGL-LGL**. Upon reaching the first intermediate group router, VLB paths can be divided into two scenarios. In scenario ④, the packet is forwarded directly to the exit router and then to G_d , with the paths being **G-L-GL/LG-L-GL**. In scenario ⑤, if an intermediate router R_i is selected for routing, the hop count increases by 1, with the paths being **GL-LGL/LGL-LGL**. Table I lists the routing hop counts and VC (virtual channel) switch counts for different scenarios of intra-group and inter-group routing.

2) *Congestion Detection:* Routers traditionally use credit-based flow control mechanism to obtain the buffer status of downstream ports, as shown in Figure 6a. This paper proposes a congestion detection method by comparing port credit counts with congestion thresholds to determine port congestion. Specifically, we define history windows W for each router's output port at the granularity of VCs. The history window tracks the number of flits sent through the output

TABLE I: Routing path types and vc switch count

Destination	Routing Type	Path type	Hops	VC Switch Count
Intra-group Router	MIN Path	L	1	0
	VLB Path	L-L	2	1
Inter-group Router	MIN Path	L-G-L	3	1
	VLB Path	G-L-GL	4	2
	VLB Path	LG-L-GL	5	2
	VLB Path	GL-LGL	5	3
	VLB Path	LGL-LGL	6	3

port's VC channel over a specific time period. For example, $W(200)$ refers to the number of flits sent in a 200-cycle period. Based on the conclusion in [24], the size of the history window is set to $2L$, where L is the link latency of the port. We calculate more accurate congestion information by subtracting the number of flits $W(2L)$ recorded in the sliding window from the actual credit usage number Q of the router's specific port. For local output ports, the function $CongL$ is used to determine congestion, with the decision logic shown in Algorithm 1.

Algorithm 1: Local Port Congestion Detection

```

1 if  $(Q_l - W(2L)) \geq Th_l$  then
2   return true;
3 else
4   return false;
```

Where Q_l represents the local output port's credit usage count (at the VC granularity), $W(2L)$ is the number of packets recorded in the history window, and Th_l is the local port's congestion threshold. If the congestion estimate $Q_l - W(2L)$ is greater than or equal to the corresponding congestion threshold, the port is deemed congested and returns true; otherwise, it returns false, indicating the port is available for use. For the global output port, the function $CongG$ is used for congestion determination. The decision logic of function $CongG$ is similar to that of $CongL$, as detailed in Algorithm 2.

Algorithm 2: Global Port Congestion Detection

```

1  $Th_g = (g \in G_B) ? Th_{g2} : Th_{g1}$ 
2 if  $(Q_g - W(2L)) \geq Th_g$  then
3   return true;
4 else
5   return false;
```

Each router uses the $CongG$ function to determine the congestion status of global port g and embeds the congestion bitmap of its global ports into the credit control packet to assist in optimizing routing decisions, as shown in Figure 6b. Each router in the group continuously broadcasts the congestion bitmap to all other routers within the group using credit-based flow control mechanism.

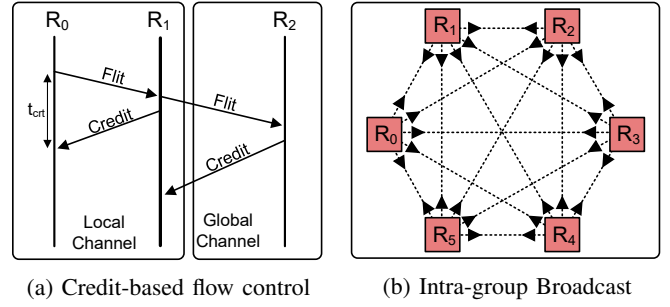


Figure 6: Credit-based flow control and intra-group broadcast for global information interaction

Algorithm 3: PIAR Algorithm

Source $C_s = \langle R_s, G_s \rangle$, Destination $C_d = \langle R_d, G_d \rangle$, Intermediate Group G_i , p_m : min port to R_d .
 L_A, L_B : port sets for intra-group traffic.
 G_A, G_B, G_C, L_D, L_E : port sets corresponding to routing scenarios ①, ②, ③, ④, and ⑤.
 W_m, W_{nm} : history window for MIN (VLB) ports.
 Fri : whether the packet first reaches G_i .

```

1 if  $R_i == R_s$  then
2   if  $G_d == G_s$  then
3     foreach  $L \in \{L_A, L_B\}$  do
4       while not_empty(L) do
5          $r = \text{random}(L)$ ;  $L- = r$ 
6         if  $\neg CongL(r)$  then
7           route through port  $r$ 
8   else
9     foreach  $G \in \{G_A, G_B, G_C\}$  do
10      while not_empty(G) do
11         $g = \text{random}(G)$ ;  $G- = g$ 
12         $r = \text{select\_port}(g)$ 
13        if not_empty(r) then
14          if  $(\neg CongL(r) \wedge \neg CongG(g))$  then
15            route through port  $r$ 
16          else
17            if  $(\neg CongG(g))$  then
18              route through port  $g$ 
19        route according to Equation 2
20    else if  $((G_i \neq G_s) \wedge (G_i \neq G_d) \wedge Fri)$  then
21      foreach  $L \in \{L_D, L_E\}$  do
22        while not_empty(L) do
23           $r = \text{random}(L)$ ;  $L- = r$ 
24          if  $r \in G$  or  $\neg CongL(r)$  then
25            route through port  $r$ 
26        route randomly
27    else
28      route MIN through port  $p_m$ 
```

3) *PIAR Algorithm*: PIAR considers both local and global port congestion, and prioritizes paths with fewer hops at the source router and intermediate group ingress, as shown in

Algorithm 3. Initially, lines 2–7 handle intra-group traffic at the source router by searching the routing table to generate a set of MIN paths, denoted as L_A , and a set of VLB paths, denoted as L_B . PIAR randomly selects a congestion-free path from L_A ; if none is available, it randomly selects a congestion-free path from L_B .

Next, lines 8–16 handle inter-group traffic. As described in IV-B1, the routing table is consulted to obtain path sets G_A , G_B , and G_C for scenarios ①, ②, and ③. PIAR prioritizes randomly selecting a congestion-free path from G_A . If none is available, it proceeds to G_B , and then to G_C . If none of these paths is available, line 17 makes routing decisions by combining UGAL with the history window method:

$$(Q_m - W_m) \times H_m \leq (Q_{nm} - W_{nm}) \times H_{nm} + T \quad (2)$$

When packets first arrive at an intermediate group, lines 18–23 obtain the shortest path ports set L_D and other path ports set L_E from the routing table, as per the scenarios ④ and ⑤ in IV-B1. The algorithm then attempts to randomly select a congestion-free path from L_D and, if none is available, from L_E . If neither provides a usable path, line 23 indicates that a path will be randomly selected as an alternative. Finally, for all other cases, the MIN routing strategy is applied.

4) Microarchitecture of the Routing Computation Unit:

Figure 7 illustrates the microarchitecture of the routing computation unit. Compared to conventional routing computation units, this unit introduces three additional subcomponents: RTA, RTB, and RTC. During routing decision-making for each packet, different subcomponents are selected for processing according to the destination node DestID and current router RouterID, with lookups performed on both MIN and VLB routing tables. Taking an example with 32 local ports and 256 global ports, each entry in the MIN routing table contains four routing records. The entries in the Node and Box regions respectively store the node ports and local ports la_i . The Group region stores both the local ports la_i and the corresponding global ports ga_i . The VLB routing table includes only Box and Group regions. Entries in the Box region store a 32-bit bitmap lb_i representing candidate local ports, while entries in the Group region store both local ports and a 256-bit bitmap gb_i representing candidate global ports.

First, if the current port is a node port and the destination node belongs to a different group, the RTA subcomponent will obtain 4 MIN paths and 256 VLB paths from the MIN and VLB tables, respectively. According to lines 8–16 in Algorithm 3, the GAP, GBP, and GCP units compute the output port sets G_A , G_B , and G_C , respectively. The PS1 unit filters available paths based on the congestion bitmap information of both local and global ports, and forwards the set of available candidate paths to the arbiter unit P1. P1 prioritizes paths with smaller hop counts to select the unique winning output. If all paths are congested, path selection will be performed using Equation 2.

Next, the RTB subcomponent looks up the corresponding local ports in the routing tables. The specific scenarios are as

follows: Scenario (i), as lines 2–7 of Algorithm 3, the current port is a node port, and the target is within the same group; Scenario (ii), as lines 18–23 of Algorithm 3, the current port is a global port, and the target points to another group (non-target group). The PS2 unit listens to the congestion bitmap information of the local ports, and the arbiter unit P2 prioritizes the direct ports. If all direct ports are congested, the arbiter unit P2 will select the unique winning output from the local ports corresponding to the VLB paths. If all paths are congested, then for Scenario (i), the path is selected using Equation 2, and for Scenario (ii), the path is randomly selected. Finally, in other cases, the RTC subcomponent selects the unique port output from the MIN routing table based on the congestion status. The VC allocation module generates the virtual channel switching results based on the routing computation results.

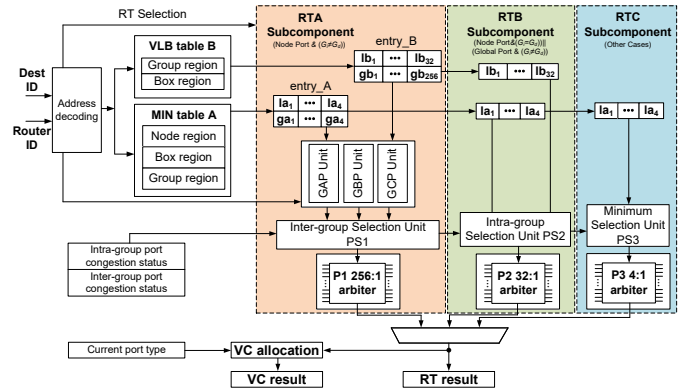


Figure 7: Microarchitecture of the routing computation unit

V. PERFORMANCE STUDY

This paper conducts extensive simulation experiments to evaluate the performance of the PIAR algorithm and compares it with advanced algorithms to gain deeper insights into the performance characteristics of PIAR. This section first provides a detailed description of the experimental methodology and simulation configurations, followed by a presentation of the specific performance analysis results.

A. Methodology

1) *Simulator and Routing Settings:* We build a cycle-accurate simulation environment based on Booksim2.0 simulator [37]. The simulator configured with single-cycle input-queued router with an internal speedup factor of 2.0, meaning the pipeline operates at twice the channel speed. The simulator uses 10,000 cycles as the sampling period. A warm-up period of 30,000 cycles is used to stabilize the network. The network is considered saturated once the average packet latency exceeds 500 cycles.

Detailed simulation parameters are summarized in Table II. Booksim2.0 provides basic implementations for MIN, VAL, and UGAL_L [9] routing algorithms. Additionally, we implemented UGAL_G [9], TPR [28], PAR_{PH} [24], and UGAL_LE [35] for performance comparisons. Virtual channels (VCs) are utilized to ensure deadlock-free routing, where

TABLE II: Simulation Parameters

Parameter	Value
Routing algorithm	MIN, VAL, UGAL_L, UGAL_G, TPR, PAR _{PH} , UGAL_LE, PIAR
Number of VCs	5 VCs: PAR _{PH} 4 VCs: VAL, UGAL_L, UGAL_G, TPR, UGAL_LE, PIAR 2 VCs: MIN
Packet size	1 flit
Buffer size	global link buffer: 256 flits local link buffer: 32 flits
Link latency	global link: 100 cycles local link: 10 cycles
History window size	global link: 200 cycles local link: 20 cycles
UGAL threshold	T : 0
Congestion threshold	Th_l : 10, Th_{g1} : 40, Th_{g2} : 10
Switch speed-up	2.0
Topology	$dflty(4,8,4,33)$

MIN uses 2 VCs, while VAL, UGAL_L, UGAL_G, TPR, UGAL_LE, and PIAR all use 4 VCs. PAR_{PH} uses 5 VCs due to additional hops within the source group. We adopt single-flit packets to avoid flow control issues. Consistent with many recent studies [24], [32], we set local channel latency to 10 cycles and global to 100 cycles. The local and global port buffers are set to 32 and 256 flits, respectively. The threshold T for UGAL is set to 0. For PIAR, the local congestion threshold (Th_l) is set to 10, while the global congestion thresholds (Th_{g1} and Th_{g2}) are set to 40 and 10, respectively. The Dragonfly topology configuration used is $dflty(4, 8, 4, 33)$, and the other scale performance curves are similar.

2) *Synthetic Traffic Patterns*: We evaluate routing algorithms using six synthetic traffic patterns: Uniform Random (UR), Adversarial (ADV+ i), Random Permutation (RP), ADV+ h , ADV+ c , and Mixed traffic. In UR traffic, packets are sent with equal probability to all destinations. Under ADV+ i traffic, nodes in group n send all traffic exclusively to group $(n + i \bmod g)$. For RP traffic, each node is randomly assigned a unique destination node. Under ADV+ c traffic, all nodes in the source group send traffic to nodes belonging to consecutive h groups. Mixed traffic is categorized by granularity into node-level (N) and router-level (R). For each, two traffic mixtures are defined: UR with ADV, and UR with RP, denoted as N/R_MIXED_UA(UR%, ADV%) and N/R_MIXED_UR(UR%, RP%), respectively, where the percentages represent the composition of each traffic type.

3) *Application Workloads*: To comprehensively evaluate the performance of routing algorithms in real applications, this paper selects six representative workloads [38] that cover a variety of communication patterns commonly found in traditional scientific computing and machine learning: *LU*, *FFT3D*, *Halo3D*, *Stencil5D*, *DL*, and *LULESH*. *LU* features **sweep** communication pattern, where processes are arranged in a 2D square and initiate network communication from one corner of the square. *FFT3D* performs **Alltoall** operations using a multi-step ring exchange. *Halo3D* and *Stencil5D* represent 3D

and 5D **stencil** patterns, respectively, where each process communicates bidirectionally with neighboring processes along all dimensions, demonstrating a typical one-to-many communication pattern. The *DL* illustrates the characteristics of **Allreduce** operations in distributed deep learning workloads. *LULESH* represents a **hybrid** communication pattern that combines non-blocking communication with collective operations, including a large amount of 3D stencil computation and 3D sweep communication. All application traffic is generated by SST [39] and injected into the Booksim2.0 simulator, ensuring that our experiments accurately reflect communication behaviors observed in production systems.

B. Results and Discussion

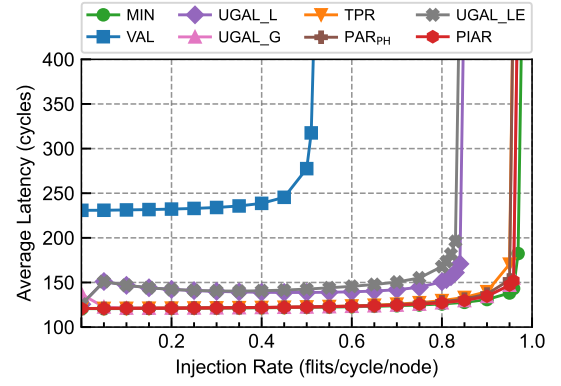


Figure 8: Uniform random (UR) traffic

1) Evaluation under Single Synthetic Traffic Patterns:

Figure 8 illustrates the performance comparison under UR traffic. The X-axis denotes the injection rates, and the Y-axis represents the average packet latency. Specifically, MIN achieves the best saturation throughput of 0.97. VAL reaches 0.51. UGAL_G, with access to global congestion information, achieves 0.96. In contrast, UGAL_L, relying solely on local information, achieves only 86.6% of MIN's throughput. PIAR closely tracks MIN's curve with a saturation throughput of 0.96. By comparison, the saturation throughputs of PAR_{PH}, TPR, and UGAL_LE are 0.95, 0.95, and 0.83 respectively, indicating that PIAR outperforms them by 1.1%, 1.1%, and 15.7%.

Figure 9 shows the performance comparison under ADV traffic pattern. In this adversarial pattern, MIN fails to balance the global link load, saturating early at an injection rate of 0.03. UGAL_L, constrained by local information, saturates at 0.12. Although UGAL_G utilizes global congestion information, its partial staleness causes saturation at 0.27. UGAL_LE provides more accurate estimation of global link congestion, reaching saturation at 0.38, a 40.7% improvement over UGAL_G. PIAR primarily selects minimal paths under low loads and favors shorter-hop paths to reduce latency while avoiding congestion. For instance, at an injection rate of 0.30, PIAR's average latency is 217.5 cycles, which is lower than VAL (240.7), PAR_{PH} (247.8), TPR (240.6), and UGAL_LE (251.1) by 9.6%,

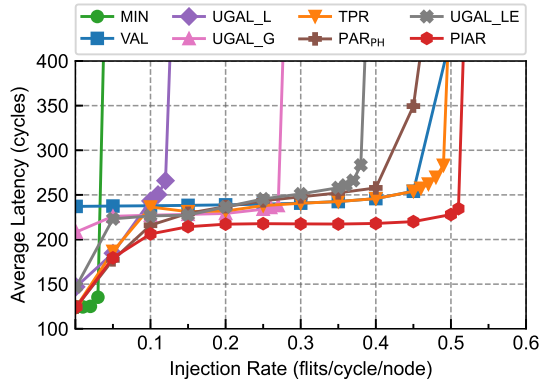


Figure 9: Adversarial (ADV) traffic

12.2%, 9.6%, and 13.4%, respectively. In terms of throughput, PIAR achieves a saturation throughput of 0.51, outperforming VAL, PAR_{PH}, TPR, and UGAL_{LE} by 2.0%, 8.5%, 4.1%, and 34.2%. Thus, PIAR demonstrates near-optimal latency at low loads and superior throughput at high loads.

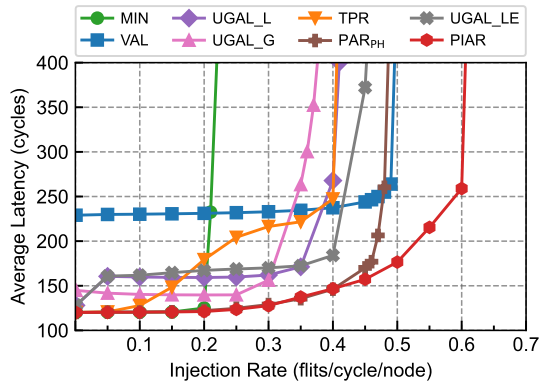


Figure 10: Random Permutation (RP) traffic

Figure 10 compares performance under RP traffic. PIAR tends to select minimal paths when the injection rate is below 0.3. As traffic increases, PIAR adapts to both local and global congestion, prioritizing shorter paths. In contrast, UGAL_{LE} and PAR_{PH} do not differentiate path lengths, consuming more resources per transmission. TPR relies on counters to infer traffic patterns, limiting its accuracy in congestion estimation. The results show saturation throughputs of 0.49, 0.48, 0.40, and 0.45 for VAL, PAR_{PH}, TPR, and UGAL_{LE}, respectively, while PIAR reaches 0.60, representing gains of 22.4%, 25.0%, 50.0%, and 33.3%. Considering the three single synthetic traffic patterns (UR, ADV, and RP), PIAR achieves an average throughput improvement of 19.2% compared to PAR_{PH}, TPR, and UGAL_{LE}, while reducing latency by up to 13.4%.

Figure 11 further analyzes PIAR's performance under ADV+*h* traffic. In this pattern, all traffic from a source group traverses the same local link in the intermediate group, increasing local link pressure and relying on intermediate routers for forwarding. At an injection rate of 0.35, PIAR achieves

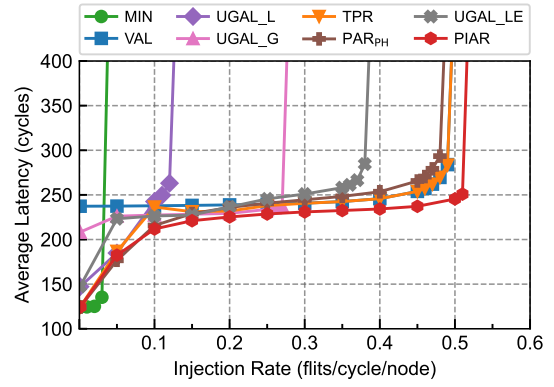


Figure 11: ADV+*h* traffic

a latency of 232.5 cycles, lower than VAL (242.5), PAR_{PH} (248.3), TPR (242.5), and UGAL_{LE} (258.1) by 4.1%, 6.4%, 4.1%, and 9.9%, respectively. Its saturation throughput reaches 0.51, outperforming the other algorithms by 4.1%, 6.3%, 4.1%, and 34.2%, respectively.

Figure 12 illustrates PIAR's performance under ADV_c traffic. In this pattern, all nodes in the source group send traffic to nodes in *h* consecutive destination groups, leading to concentration on a single router within the source group. At an injection rate of 0.40, PIAR achieves a latency of 222.1 cycles, better than VAL (246.0), PAR_{PH} (257.5), and UGAL_{LE} (238.4), with latency reductions of 9.7%, 13.7%, and 6.8%, respectively. PIAR's saturation throughput is 0.53, while VAL, PAR_{PH}, and UGAL_{LE} achieve 0.50, 0.40, and 0.47, corresponding to improvements of 6.0%, 32.5%, and 12.8%, respectively.

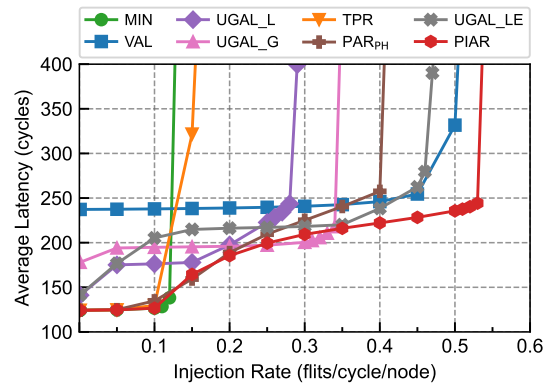


Figure 12: ADV_c traffic

2) *Evaluation under Mixed Traffic Patterns:* Figures 13 illustrates the performance of various routing algorithms under the N_MIXED_UA traffic, where the ADV traffic ratios are set to 25%, 50%, and 75%. Overall, as the ADV ratio increases, the rate of latency growth in the early region of the performance curve accelerates significantly. In Figure 13(a), when the ADV ratio is only 25%, a large portion of packets are still delivered via MIN paths, resulting in relatively balanced inter-

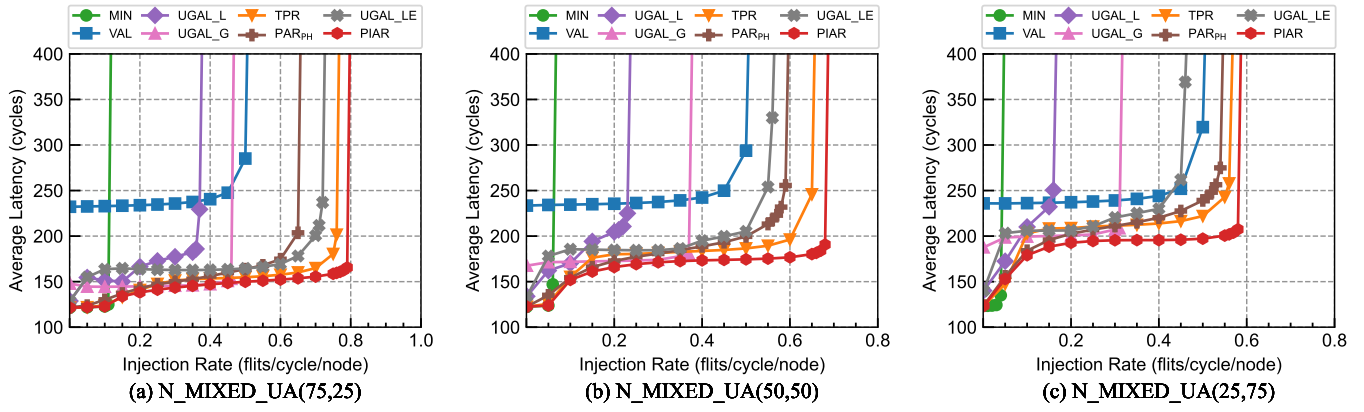


Figure 13: Node-level mixed traffic (UR + ADV)

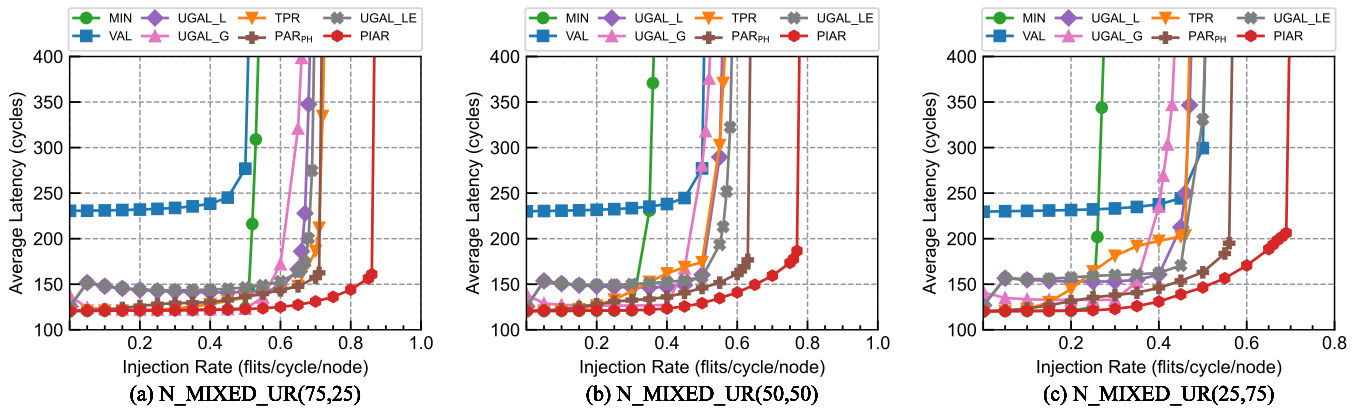


Figure 14: Node-level mixed traffic (UR + RP)

group traffic and slower latency growth. However, as the ADV ratio rises, VLB paths are used more frequently, leading to a sharp rise in latency. For example, in Figure 13(a), as the injection rate increases from 0 to 0.2, the average packet latency of PIAR increases from 121.4 to 138.2 cycles; whereas in Figure 13(c), under the same injection rate, the latency surges from 123.4 to 192.7 cycles. In the mid-load region, PIAR tends to select VLB paths with fewer hops, effectively mitigating the latency increase caused by longer paths. At an injection rate of 0.4, PIAR's average latency is 147.1, 173.4, and 195.6 cycles for ADV ratios of 25%, 50%, and 75%, respectively. Compared to PAR_{PH} , these represent reductions of 5.8%, 8.1%, and 11.2%; compared to TPR, reductions of 4.1%, 5.6%, and 8.6%; and compared to UGAL_LE, reductions of 9.7%, 11.1%, and 15.0%, respectively. In the saturation region, PIAR maintains higher throughput due to its preference for paths with fewer hops and lower resource consumption, achieving 0.79, 0.68, and 0.58 for the three ADV ratios. In contrast, although PAR_{PH} prioritizes MIN paths, it randomly selects VLB paths, resulting in lower throughputs of 0.65, 0.59, and 0.54. TPR relies on counters to estimate intra-/inter-group traffic but does not optimize VLB selection, yielding throughputs of 0.76, 0.65, and 0.57. UGAL_LE, while improving congestion estimation,

also adopts random VLB selection and achieves 0.72, 0.56, and 0.46. Overall, the throughput of PIAR is on average 12.4% higher than that of PAR_{PH} , TPR, and UGAL_LE.

Figures 14 presents the performance results under the N_MIXED_UR traffic. In the early phase of the curve, since the RP traffic does not exhibit strong adversarial characteristics, most packets still follow MIN paths. For instance, in Figure 14(b), when RP traffic is 50% and the injection rate is 0.1, the average latencies of UGAL_G, PAR_{PH} , and PIAR are 127.3, 122.5, and 120.8 cycles, respectively. In the mid-load region, the adversarial nature of RP traffic gradually intensifies. At an injection rate of 0.4 with RP ratios of 25%, 50%, and 75%, PIAR achieves average latencies of 121.9, 123.3, and 130.9 cycles, representing reductions of 6.5%, 9.3%, and 10.3% compared to PAR_{PH} ; 4.9%, 23.8%, and 33.8% compared to TPR; and 15.5%, 19.1%, and 19.4% compared to UGAL_LE. In the saturation region, as the RP ratio increases, more packets are routed via VLB paths. The throughput of PAR_{PH} , TPR, and UGAL_LE is 0.71, 0.63, and 0.56; 0.72, 0.57, and 0.48; and 0.69, 0.58, and 0.51. In contrast, PIAR achieves higher throughputs of 0.86, 0.77 and 0.69, resulting in average improvement of 28.6% over PAR_{PH} , TPR, and UGAL_LE.

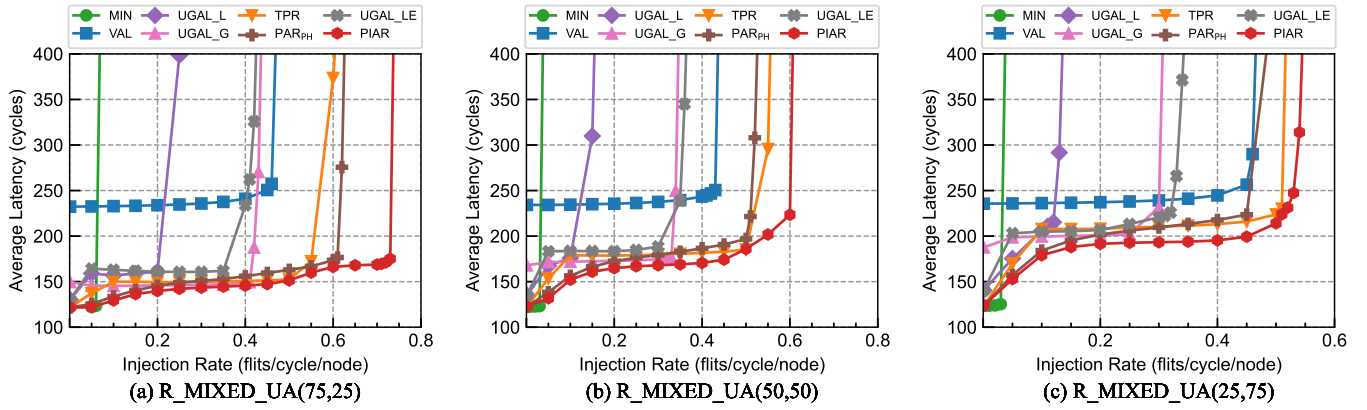


Figure 15: Router-level mixed traffic (UR + ADV)

Figure 15 shows the performance results under the R_MIXED_UA traffic. In the early region, increasing ADV ratios cause greater pressure on inter-group channels, leading to latency increases. In the mid-load region, PIAR continues to maintain its latency advantage. For instance, at an injection rate of 0.3, PIAR's average latency is 143.4, 167.9, and 193.2 cycles under ADV ratios of 25%, 50%, and 75%, representing reductions of 5.0%, 6.4%, and 7.8% compared to PAR_{PH}; 4.3%, 6.6%, and 8.1% compared to TPR; and 10.8%, 10.9%, and 12.4% compared to UGAL_{LE}. In the saturation region, PIAR achieves throughput values of 0.73, 0.60, and 0.54, compared to 0.62, 0.53, and 0.50 for PAR_{PH}; 0.62, 0.55, and 0.51 for TPR; and 0.43, 0.36, and 0.34 for UGAL_{LE}. Thus, PIAR improves throughput by an average of 29.7% over PAR_{PH}, TPR, and UGAL_{LE}.

Under mixed traffic patterns, PIAR consistently demonstrates strong adaptability. On average, it improves throughput by 23.6% over PAR_{PH}, TPR, and UGAL_{LE}, while reducing average packet latency by up to 33.8%.

3) *Evaluation under Application Workloads*: To enable a clear comparison of latency performance across different routing algorithms under various application workloads, this paper normalizes the average communication latency of each algorithm relative to PIAR, using the same network configuration as described in V-A1. The results are presented in Figure 16. In the *LU* application, PIAR (red bars) achieves the lowest latency, significantly outperforming TPR and UGAL_{LE}. In *FFT3D*, where traffic follows a pronounced AlltoAll pattern with more dispersed communication paths, PIAR still slightly outperforms PAR_{PH}. In *Halo3D*, which involves intensive neighbor exchanges across three dimensions, PIAR achieves the lowest latency, outperforming PAR_{PH} by approximately 14.6%. For *Stencil5D*, which features high-dimensional neighborhood communication, PIAR maintains its advantage in path selection under high traffic density, achieving slightly better latency than PAR_{PH}. In the *DL* workload, PIAR significantly outperforms both UGAL_G and UGAL_{LE}. In the *LULESH* application, which involves a complex mixture of communication modes, PIAR remains competitive, delivering lower

latency than TPR, PAR_{PH}, and UGAL_{LE}, and only slightly higher than UGAL_G. In summary, PIAR delivers optimal or near-optimal latency performance on all six application workloads, reducing the latency by 24.0% on average compared to PAR_{PH}, TPR, and UGAL_{LE}, demonstrating its broad applicability and robustness under diverse communication patterns.

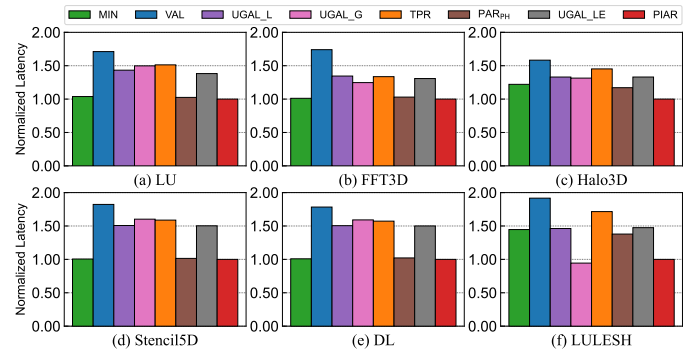


Figure 16: Evaluation results on application workloads

VI. CONCLUSION

In this work, we propose PIAR — **Path-Improved Adaptive Routing** algorithm. PIAR dynamically selects routing paths based on the status of local and global channels, and prioritizes non-minimal paths with fewer hops to reduce network latency and load. Extensive experimental results under the single synthetic traffic patterns, mixed traffic patterns and application workloads demonstrate that PIAR significantly outperforms state-of-the-art algorithms, achieving lower average network latency and notably higher throughput, thereby validating the effectiveness of PIAR in addressing the challenges of adaptive routing in Dragonfly networks.

ACKNOWLEDGMENT

This research is supported by the National Key Research and Development Program of China (2023YFB4403400).

REFERENCES

- [1] S. Yamamura, Y. Akizuki, H. Sekiguchi, T. Maruyama, T. Sano, H. Miyazaki, and T. Yoshida, "A64fx: 52-core processor designed for the 442petaflops supercomputer fugaku," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65. IEEE, 2022, pp. 352–354.
- [2] M. Al-Fares, A. Loukissas, and A. Vahdat, "A scalable, commodity data center network architecture," *ACM SIGCOMM computer communication review*, vol. 38, no. 4, pp. 63–74, 2008.
- [3] D. De Sensi, S. Di Girolamo, K. H. McMahon, D. Roweth, and T. Hoefler, "An in-depth analysis of the slingshot interconnect," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–14.
- [4] J. Cao, L. Xiao, Z. Pang, K. Wang, and J. Xu, "The efficient in-band management for interconnect network in tianhe-2 system," in *2016 24th Euromicro International Conference on Parallel, Distributed, and Network-Based Processing (PDP)*. IEEE, 2016, pp. 18–26.
- [5] J. Gao, F. Zheng, F. Qi, Y. Ding, H. Li, H. Lu, W. He, H. Wei, L. Jin, X. Liu *et al.*, "Sunway supercomputer architecture towards exascale computing: analysis and practice," *Science China Information Sciences*, vol. 64, no. 4, p. 141101, 2021.
- [6] J. Kim, W. J. Dally, B. Towles, and A. K. Gupta, "Microarchitecture of a high radix router," in *32nd International Symposium on Computer Architecture (ISCA'05)*. IEEE, 2005, pp. 420–431.
- [7] J. Kim, W. J. Dally, and D. Abts, "Flattened butterfly: a cost-efficient topology for high-radix networks," in *Proceedings of the 34th annual international symposium on Computer architecture*, 2007, pp. 126–137.
- [8] J. H. Ahn, N. Binkert, A. Davis, M. McLaren, and R. S. Schreiber, "Hyperx: topology, routing, and packaging of efficient large-scale networks," in *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis*, 2009, pp. 1–11.
- [9] J. Kim, W. J. Dally, S. Scott, and D. Abts, "Technology-driven, highly-scalable dragonfly topology," in *2008 International Symposium on Computer Architecture*. IEEE, 2008, pp. 77–88.
- [10] A. Shpiner, Z. Harnamty, S. Eliad, V. Zdornov, B. Gafni, and E. Zahavi, "Dragonfly+: Low cost topology for scaling datacenters," in *2017 IEEE 3rd International Workshop on High-Performance Interconnection Networks in the Exascale and Big-Data Era (HiPINEB)*. IEEE, 2017, pp. 1–8.
- [11] G. Faanes, A. Bataineh, D. Roweth, T. Court, E. Froese, B. Alverson, T. Johnson, J. Kopnick, M. Higgins, and J. Reinhard, "Cray cascade: a scalable hpc system based on a dragonfly network," in *SC'12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE, 2012, pp. 1–9.
- [12] M. Besta and T. Hoefler, "Slim fly: A cost effective low-diameter network topology," in *SC'14: proceedings of the international conference for high performance computing, networking, storage and analysis*. IEEE, 2014, pp. 348–359.
- [13] M. Flajslik, E. Borch, and M. A. Parker, "Megafly: A topology for exascale systems," in *High Performance Computing: 33rd International Conference, ISC High Performance 2018, Frankfurt, Germany, June 24-28, 2018, Proceedings 33*. Springer, 2018, pp. 289–310.
- [14] F. Lei, D. Dong, X. Liao, X. Su, and C. Li, "Galaxyfly: A novel family of flexible-radix low-diameter topologies for large-scales interconnection networks," in *Proceedings of the 2016 International Conference on Supercomputing*, 2016, pp. 1–12.
- [15] K. Lakhota, M. Besta, L. Monroe, K. Isham, P. Iff, T. Hoefler, and F. Petrini, "Polarfly: a cost-effective and flexible low-diameter topology," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [16] top500.org, "TOP500 List – November 2024," <https://www.top500.org/lists/top500/2024/11/>, accessed: 2025-04-22.
- [17] L. G. Valiant and G. J. Brebner, "Universal schemes for parallel communication," in *Proceedings of the thirteenth annual ACM symposium on Theory of computing*, 1981, pp. 263–277.
- [18] A. Singh, *Load-balanced routing in interconnection networks*. Stanford University, 2005.
- [19] E. Hastings, D. Rincon-Cruz, M. Spehlmann, S. Meyers, A. Xu, D. P. Bunde, and V. J. Leung, "Comparing global link arrangements for dragonfly networks," in *2015 IEEE International Conference on Cluster Computing*. IEEE, 2015, pp. 361–370.
- [20] Z. S. A. Alzaid, S. Bhowmik, X. Yuan, and M. Lang, "Global link arrangement for practical dragonfly," in *Proceedings of the 34th ACM International Conference on Supercomputing*, 2020, pp. 1–11.
- [21] P. Fuentes, E. Vallejo, C. Camarero, R. Beivide, and M. Valero, "Network unfairness in dragonfly topologies," *The Journal of Supercomputing*, vol. 72, pp. 4468–4496, 2016.
- [22] M. García, E. Vallejo, R. Beivide, M. Odriozola, C. Camarero, M. Valero, G. Rodríguez, J. Labarta, and C. Minkenberg, "On-the-fly adaptive routing in high-radix hierarchical networks," in *2012 41st International Conference on Parallel Processing*. IEEE, 2012, pp. 279–288.
- [23] N. Jiang, J. Kim, and W. J. Dally, "Indirect adaptive routing on large scale interconnection networks," in *Proceedings of the 36th annual international symposium on Computer architecture*, 2009, pp. 220–231.
- [24] J. Won, G. Kim, J. Kim, T. Jiang, M. Parker, and S. Scott, "Overcoming far-end congestion in large-scale networks," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2015, pp. 415–427.
- [25] M. García, E. Vallejo, R. Beivide, M. Valero, and G. Rodríguez, "Ofarc: Efficient dragonfly networks with simple congestion management," in *2013 IEEE 21st Annual Symposium on High-Performance Interconnects*. IEEE, 2013, pp. 55–62.
- [26] M. García, E. Vallejo, R. Beivide, M. Odriozola, and M. Valero, "Efficient routing mechanisms for dragonfly networks," in *2013 42nd International Conference on Parallel Processing*. IEEE, 2013, pp. 582–592.
- [27] P. Fuentes, E. Vallejo, M. García, R. Beivide, G. Rodríguez, C. Minkenberg, and M. Valero, "Contention-based nonminimal adaptive routing in high-radix networks," in *2015 IEEE International Parallel and Distributed Processing Symposium*. IEEE, 2015, pp. 103–112.
- [28] P. Faizian, J. F. Alfaro, M. S. Rahman, M. A. Mollah, X. Yuan, S. Pakin, and M. Lang, "Tpr: traffic pattern-based adaptive routing for dragonfly networks," *IEEE Transactions on Multi-Scale Computing Systems*, vol. 4, no. 4, pp. 931–943, 2018.
- [29] M. S. Rahman, S. Bhowmik, Y. Ryasianskiy, X. Yuan, and M. Lang, "Topology-custom ugal routing on dragonfly," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2019, pp. 1–15.
- [30] M. Benito, P. Fuentes, E. Vallejo, and R. Beivide, "Acor: Adaptive congestion-oblivious routing in dragonfly networks," *Journal of Parallel and Distributed Computing*, vol. 131, pp. 173–188, 2019.
- [31] M. Benito, E. Vallejo, and R. Beivide, "Lia: Latency-improved adaptive routing for dragonfly networks," *ACM Transactions on Architecture and Code Optimization*, 2025.
- [32] H. Kasan, G. Kim, Y. Yi, and J. Kim, "Dynamic global adaptive routing in high-radix networks," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 771–783.
- [33] Y. Kang, X. Wang, and Z. Lan, "Q-adaptive: A multi-agent reinforcement learning based routing on dragonfly network," in *Proceedings of the 30th International Symposium on High-Performance Parallel and Distributed Computing*, 2021, pp. 189–200.
- [34] R. S. Chaulagain, F. T. Liza, S. Chunduri, X. Yuan, and M. Lang, "Achieving the performance of global adaptive routing using local information on dragonfly through deep learning," *ACM/IEEE SC tech poster*, 2020.
- [35] R. S. Chaulagain and X. Yuan, "Enhanced ugal routing schemes for dragonfly networks," in *Proceedings of the 38th ACM International Conference on Supercomputing*, 2024, pp. 449–459.
- [36] W. J. Dally and B. P. Towles, *Principles and practices of interconnection networks*. Elsevier, 2004.
- [37] N. Jiang, D. U. Becker, G. Michelogiannakis, J. Balfour, B. Towles, D. E. Shaw, J. Kim, and W. J. Dally, "A detailed and flexible cycle-accurate network-on-chip simulator," in *2013 IEEE international symposium on performance analysis of systems and software (ISPASS)*. IEEE, 2013, pp. 86–96.
- [38] Y. Kang, X. Wang, and Z. Lan, "Study of workload interference with intelligent routing on dragonfly," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–14.
- [39] A. F. Rodrigues, K. S. Hemmert, B. W. Barrett, C. Kersey, R. Oldfield, M. Weston, R. Risen, J. Cook, P. Rosenfeld, E. Cooper-Balis *et al.*, "The structural simulation toolkit," *ACM SIGMETRICS Performance Evaluation Review*, vol. 38, no. 4, pp. 37–42, 2011.